

This document follows on from `Compare_to_ZZ.md` and supersedes `Compare_to_ZZ_and_Recommend_02.md` . That comparison concluded that ZZ’s harness and Test_Bed’s four-tool bundle overlap on turning a plain-language ask into spec → tests → code → review, but ZZ adds six things Test_Bed has no counterpart for: a business-owner sign-off state machine, a golden-test CI merge gate, risk-tiered tool permissions enforced in config, a fixed business-facing PR template, a business-trial loop distinct from code review, and a cross-project reusable-capability registry — plus deployment/compliance choreography that is genuinely out of scope for a skills package.

This document turns that gap list into **concrete file-level changes to Test_Bed/** , each one something a Fellow can actually add to the repo — templates, a `settings.json` , hooks, a CI workflow — not a restatement of the gap. Every recommendation is scoped to what a self-serve tooling repo can reasonably own; the purely organizational parts of ZZ’s proposal (10-person staffing, named delivery leads, phased client engagements) are called out at the end as **deliberately not imported**, with a note on what a solo or two-person Fellow team should do instead.

What changed from _02 : \$0 is rewritten again after actually running the install on 2026-08-19. `claude plugin install` was run for all three plugins and `claude plugin list` now confirms all three installed and enabled at user scope. The install gap that blocked `_01` and `_02` is closed. The one thing still unverified is whether the resulting skills/commands actually *fire* in a live session — see \$0 for what that means and why it’s a lighter caveat than a missing install. Everything else in this document is unchanged from `_02` , apart from removing “once installed” hedges that no longer apply.

0. Priority zero, resolved: all three plugins installed and enabled

`Compare_to_ZZ.md` \$1 originally found that **three of the four frameworks Test_Bed recommends were not installed in this repo**. `_01` flagged it; `_02` found the marketplaces registered but the plugins still not installed. As of 2026-08-19, the install itself has been run and confirmed:

```
claude plugin install superpowers@superpowers-marketplace -y
claude plugin install compound-engineering@compound-engineering-plugin -y
claude plugin install andrej-karpathy-skills@karpathy-skills -y
```

`claude plugin list` confirms:

Plugin	Version	Scope	Status
superpowers@superpowers-marketplace	6.3.0	user	✓ enabled
compound-engineering@compound-engineering-plugin	3.22.4	user	✓ enabled
andrej-karpathy-skills@karpathy-skills	1.0.0	user	✓ enabled

This closes the gap `_01` and `_02` both flagged as priority zero. Every recommendation below can now assume Superpowers, Compound Engineering, and karpathy-skills are installed — R4 in particular,

which depends on `/ce-code-review` and Superpowers' TDD gates existing at all.

One remaining, lighter caveat: installed-and-enabled (per `claude plugin list`) is a different claim than "actually fires correctly in a running session." The session used to run the install predates it, so it couldn't show the new `/ce-*` commands, Superpowers skills, or the karpathy-guidelines skill in its own available-tools list — that's expected, not a sign of failure, since a session's command/skill list is established at start. This is not the same open question `_02` raised (registered-but-uninstalled); it's the normal "restart to pick up a new install" step any plugin/extension system has. **Action:** in a fresh Claude Code session in this repo, check `/plugins` (should list all three), and confirm `/ce-code-review`, `/ce-plan`, `/ce-work`, `/ce-compound` autocomplete, and that a Superpowers-driven TDD skill and the karpathy-guidelines skill appear in that session's skill listing. This is a one-time, low-effort check — do it before relying on R4's claim that Compound's review and Superpowers' TDD gates are "actually firing," but it no longer blocks treating the install itself as done.

1. Recommendations, in order of leverage

Each recommendation names the ZZ mechanism it's replicating, the concrete file(s) to add, and — since the user asked specifically for **simple advice a human can read** — a short paragraph written the way `README.md` / `SE_Discipline.md` already talk to a non-expert user.

R1 — A business-context-bearing project file (ZZ's `AGENTS.md` schema)

ZZ mechanism: a committed context file with six mandated sections (business context, process/data, rules, technical, tests/commands, risk boundaries) that the AI reads before every task — not generic style guidance, but the actual business rules.

Gap today: `Test_Bed/CLAUDE.md` and `SE_Discipline.md` are entirely process guidance (which tool to run when). Nothing captures *this project's* business rules, data shapes, or risk tiers. A new project cloned from `Test_Bed` starts with zero business memory.

Concrete change: add a template file `PROJECT_CONTEXT.md.template` at the repo root, six sections, modeled directly on ZZ's schema:

PROJECT_CONTEXT.md – [project name]

Claude: read this alongside SE_Discipline.md before any work. Human: fill in every section below before the first `/openspec-propose`. Blank sections mean Claude cannot tell business-correct from merely-plausible.

1. Business context

Project goal · who uses this · what pain point it removes · who owns the answer when Claude is unsure (name, not a role title).

2. Process / data

The real-world flow this system automates, step by step. Input/output shapes, field meanings, sample records, which fields are sensitive.

3. Rules

Business rules, edge-case rules, rejection rules, manual-override rules – in plain language, one bullet per rule. If a rule lives only in someone's head today, write it here first.

4. Technical

Architecture, code layout, interface conventions, data-store design – only what's decided; leave undecided things marked "TBD, ask before assuming."

5. Tests / commands

How to run tests, where golden cases live (see R3), what "passing" means, regression scope.

6. Risk boundaries

Three lists: ****AI may edit directly**** · ****AI may edit, human must confirm**** · ****off-limits to AI**** (see R4 – this section should mirror `.claude/settings.json`, not just describe it).

Wire it in by adding one line to CLAUDE.md's "read this first"

list: @PROJECT_CONTEXT.md alongside @SE_Discipline.md. SE_Discipline.md Step 0 (Triage) should add: *"If PROJECT_CONTEXT.md doesn't exist or Section 3/6 are empty, treat every request as Unclear — ask, don't guess."*

Human-facing guideline (add to README Part B): *"Before you ask Claude to build anything, spend 15 minutes filling in PROJECT_CONTEXT.md — especially the Rules and Risk boundaries sections. This is the single highest-leverage thing you can do: it's the difference between Claude asking you the right clarifying question and Claude quietly guessing wrong."*

R2 — A requirement-ticket template with a named owner and a real freeze gate

ZZ mechanism: every requirement passes through five states — **Submit** → **AI check** → **Review** → **Confirm** → **Freeze** — via a structured ticket with four mandatory fields (current process, problems, expected solution, acceptance criteria). "No frozen baseline, no build starts" is a hard gate, and the AI-check step produces a blocking action list (ZZ's concrete example: 13 items requiring Finance confirmation before build could start).

Gap today: `/openspec-propose` drafts a spec, then SE_Discipline.md Step 1 says "stop and show the human... ask: approve or adjust." That's a single informal checkpoint, not a state machine, and nothing forces the *business* fields (current process / problems / expected solution / acceptance criteria) into the proposal, nor produces a blocking action list of open questions before build starts.

Concrete change: add `openspec/templates/requirement-ticket.md`, and reference it from `openspec/config.yaml`'s `rules.proposal` so `/openspec-propose` is instructed to fill it as part of `proposal.md` rather than free-form:

```
## Requirement ticket
| Field | Content |
|---|---|
| Current process | How this is done today (manually or in the old system) |
| Problems | What's wrong with the current process – be specific, not "it's slow" |
| Expected solution | What the new system should do instead |
| Acceptance criteria | The concrete, checkable bar for "this is done" |

## Open items (AI-check output)
List every missing fact, unstated assumption, and risk this proposal surfaces. Mark each:
Blocks build / Blocks acceptance only / Not this version. Do not proceed past this
proposal to /ce-plan while any "Blocks build" item is unresolved.

## Requirement owner
Name: ____ ← a person, not "the team." This person's approval is the Freeze in Step 5.
```

Update `openspec/config.yaml`:

```
rules:
  proposal:
    - Fill the Requirement ticket table before writing scope or design.
    - Populate Open items as a blocking/non-blocking list; do not silently resolve ambiguity.
    - A named Requirement owner must be recorded before the proposal is considered frozen.
```

Human-facing guideline: *"When Claude shows you a proposal to approve, check two things before you say yes: (1) is there a named owner next to 'Requirement owner' — if it just says 'the team,' push back and name a person; (2) are there any 'Blocks build' items still open — if so, the proposal isn't frozen yet, it's a draft."*

R3 — Golden-test assets + a CI gate that can't be talked past

ZZ mechanism: 5–10 golden cases per capability (input + expected output, including intermediate steps, not just the final figure), plus an exception/edge-case list, built *before* any feature code, and a merge gate: tests fail → code cannot merge, enforced by tooling, plus a weekly re-run of sampled production traces to catch drift.

Gap today: no tests/ convention, no CI, no merge-blocking mechanism anywhere in `Test_Bed`. `SE_Discipline.md`'s TDD step (Superpowers, now installed per §0) enforces *unit-level* red→green→refactor, but there is still no concept of a business-confirmed golden sample or a threshold that blocks merge regardless of unit-test status.

Concrete change: three pieces —

1. **Convention:** a `tests/golden/<capability>/` folder per capability, each case as `NN-description.input.json + NN-description.expected.json`, plus `tests/golden/EXCEPTIONS.md` for the edge-case list (mirroring ZZ's five risk classes: bad/missing data, ordering violations, unauthorized parameter changes, permission violations, out-of-bound values — adapt names to the project's domain).
2. **CI workflow** — `.github/workflows/golden-gate.yml` :

```
name: golden-gate
on: [pull_request]
jobs:
  golden-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run golden + regression suite
        run: npm test -- --grep "golden|regression"
      - name: Enforce minimum golden coverage
        run: node scripts/check-golden-coverage.js --min-per-capability 5
```

(adapt the test command to the project's actual stack — the point is the job exists and is a **required check** on the PR, not that it's this exact command).

3. **A weekly drift job** — `.github/workflows/golden-drift-check.yml` on a `schedule: cron`, re-running the golden + regression suite against a sampled recent trace and opening an issue on failure.

Human-facing guideline (add to README): *"Every capability needs 5–10 golden cases — real examples with a known-correct answer — before Claude starts building it, not after. If you can't produce 5 confirmed examples, that's a sign the requirement isn't actually clear yet; go back to R2's ticket, don't skip to code."*

R4 — Risk-tiered permissions enforced in `.claude/settings.json`, plus hooks

ZZ mechanism: four tiers (Low/Medium/High/Off-limits) mapped to handling rules, enforced through MCP server scoping and `settings.json` deny rules — not a prompt instruction the model can ignore — plus hooks that block a commit when tests haven't passed and require human confirmation before edits to permission/audit directories.

Gap today: the repo has a `.claude/settings.local.json` (permissions only — `Bash(claude plugin *)` and a read allow-rule), but no risk-tiered `deny / ask` rules and no hooks anywhere in `Test_Bed.karpathy-skills` "surgical changes" guardrail is now installed (\$0), but remains a *behavioral* rule the model follows voluntarily, not a config-enforced boundary — installing the skill doesn't change that distinction, only `settings.json` deny-rules do.

Concrete change: add `.claude/settings.json.template` :

```

{
  "permissions": {
    "allow": [
      "Edit(src/ui/**)",
      "Edit(src/utls/**)"
    ],
    "ask": [
      "Edit(src/rules/**)",
      "Edit(src/validation/**)",
      "Edit(**/*.audit*)"
    ],
    "deny": [
      "Edit(secrets/**)",
      "Edit(.env*)",
      "Bash(*prod*)",
      "Read(**/prod_data/**)"
    ]
  },
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit",
        "hooks": [{ "type": "command", "command": "scripts/warn-if-audit-path.sh" }]
      }
    ]
  }
}

```

Pair this with a short `RISK_TIERS.md` at the repo root that mirrors ZZ’s table and tells the Fellow which project directories to sort into which tier — this is the file a human actually edits; `settings.json` is generated from it, the way `PROJECT_CONTEXT.md` §6 should match it (see R1).

Tier	Example	Handling
Low	UI copy, styling, pure utility functions	AI edits directly, normal review
Medium	Business flow, data validation, report logic	Test coverage required + human review
High	Calc rules, permissions, audit, sensitive data	Tech-lead/human review required, no auto-merge
Off-limits	Prod data, secrets, deploy config, core permissions	AI must not touch — deny-listed, not just discouraged

Also add the enforcement half of “Plan → Execute → Verify” that ZZ’s hooks provide and `Test_Bed` currently leaves to Superpowers’ voluntary skill-trigger: a `PreToolUse` hook on `Bash(*commit*)` that refuses to run unless the last recorded test run passed, and a `PreToolUse` hook on `Edit` matching the “ask” tier paths above that forces a stop-and-confirm even mid-session.

Human-facing guideline: “*RISK_TIERS.md* is the one file where you’re drawing a line, not writing prose: which folders can Claude touch freely, which need your review, and which it should never touch at all. Fill

it in once per project, right after `PROJECT_CONTEXT.md`. Karpathy's guardrails are installed now, but they're still advisory — a config-enforced deny-list is the difference between 'the model tries not to' and 'it structurally can't'."

R5 — A fixed PR template with business questions

ZZ mechanism: every submission answers four fixed questions: which business rule / test case does this serve; does it touch calc/permissions/audit/data; does it break existing flows, and what's the rollback plan; what's AI-generated vs. human-reviewed.

Gap today: NO `.github/PULL_REQUEST_TEMPLATE.md` in Test_Bed; `/ce-code-review` (installed per §0) covers security/architecture/quality, none of which are business-facing questions.

Concrete change: add `.github/PULL_REQUEST_TEMPLATE.md` :

```
## What business rule or requirement ticket does this serve?
```

```
(link the ticket from R2 – a PR with no linked ticket should be rare and explained)
```

```
## Does this touch calc / permissions / audit / sensitive data?
```

- [] Yes – High-risk tier (R4), needs human sign-off before merge
- [] No

```
## Golden / regression coverage
```

- [] New or changed golden cases included for this change
- [] Full regression suite passes (see CI check)

```
## Rollback plan
```

```
(one sentence – how do we undo this if it's wrong in production?)
```

```
## AI-generated vs. human-reviewed
```

```
Which parts were Claude-authored, which were hand-written or hand-edited, and who reviewed the diff line-by-line.
```

Human-facing guideline: *"Don't approve a merge just because the CI checkbox is green. Read the 'does this touch calc/permissions/audit/data' answer — if yes, that PR needs your eyes on the actual diff, not just the test output."*

R6 — A business-trial / decision-log loop, distinct from code review

ZZ mechanism: Stage 4 of the six-stage pipeline. The business *uses* the real thing and judges fast whether it matches real operations; feedback enters a loop — trial → decision log → backlog → AI edit → test — never scattered in chat. Critically: *every* piece of feedback becomes a new test case before the code changes (ZZ Part 3 Step 6: "one fix, fixed for good").

Gap today: `/ce-code-review` (installed per §0) is an *engineer's* review (security/architecture/quality). Nothing in Test_Bed represents "the business person clicks around and tells us what's wrong," and nothing forces feedback to become a test before it becomes a code change.

Concrete change: insert a new step between today's Step 4 (Review) and Step 5 (Compound) in `SE_Discipline.md`, and add `DECISION_LOG.md` template:

```
# DECISION_LOG.md – [change name]
One row per piece of business-trial feedback. A row without a linked test case is not yet
actionable – go add the test case first.

| Date | Raised by | What they saw vs. expected | New test case added? | Resolution |
|---|---|---|---|---|
```

`SE_Discipline.md` addition:

Step 4.5 — Business trial (human checkpoint, distinct from code review). Before final merge on anything Substantial, ask the business owner (named in R2) to actually use the change against a realistic scenario — not read a diff. Log every reaction in `DECISION_LOG.md`. **Convert each item to a golden case or exception test (R3) before touching code again** — do not patch based on a verbal description alone.

Human-facing guideline: “Code review answers ‘is this well-built.’ The business trial answers ‘is this actually right.’ Don’t skip straight from ‘tests pass’ to ‘ship it’ — have the person who owns the business process try it first, even for five minutes.”

R7 — A capability registry that compounds across projects

ZZ mechanism: every stage banks a reusable asset; the value isn’t one project, it’s compounding — a second project (Trading) explicitly reuses the first project’s (PM comps) approval flow, audit log, and validation layer rather than rebuilding them. This is a literal, named registry, not just git history.

Gap today: OpenSpec’s `openspec/changes/archive/` keeps a dated record of finished changes, but doesn’t extract or catalog *reusable components* for the next project. Compound Engineering’s `/ce-compound` (installed per §0) documents learnings (process knowledge) but not a component catalog (code assets).

Concrete change: add `CAPABILITY_REGISTRY.md` at the repo root, and extend `SE_Discipline.md` Step 5 (Compound) to update it as a required action, not optional:

```
# CAPABILITY_REGISTRY.md
Reusable components banked by completed changes in this repo. Before starting a new
`/openspec-propose`, check this list – reuse before rebuilding.

| Component | What it does | Added by (change) | Where it lives |
|---|---|---|---|
| Rules engine | Evaluates named business rules against a record | add-pm-comps | src/rules/ |
| Approval flow | Two-person maker/checker sign-off | add-pm-comps | src/approval/ |
| Audit Log | Append-only change log with actor/timestamp | add-pm-comps | src/audit/ |
```

SE_Discipline.md Step 5 addition: " /ce-compound must also ask: did this change produce anything a future project could reuse wholesale (not just learnings, but an actual component)? If yes, add a row to CAPABILITY_REGISTRY.md before archiving."

Human-facing guideline: "Before you describe a new feature to Claude, glance at CAPABILITY_REGISTRY.md — if something similar already exists, say so ('reuse the approval flow from last time') instead of letting Claude build it fresh. This is what makes project 3 faster than project 1."

R8 — A lightweight deployment / data-governance checklist (guidance only)

ZZ mechanism: on-prem-only deployment, masked-data test environment, two-person approval for live parameter entry, full audit logging — genuine infrastructure/compliance choreography.

Gap today: nothing, and this is correctly out of scope for a generic skills package — Test_Bed cannot provision Dymon's network or IT approvals. But Fellows still need to know these questions exist *before* they hit central IT with a PROD promotion request, which is Test_Bed's stated mission (WHAT_THIS_DIR_IS.md / README.md).

Concrete change: add DEPLOYMENT_CHECKLIST.md as a **guidance-only, no-tooling** template — a list of questions to answer before requesting a PROD promotion, not something Claude executes:

```
# DEPLOYMENT_CHECKLIST.md – fill this in before asking IT to promote to PROD
- [ ] Where does this run – on-prem / Dymon's own environment, or does data leave the boundary?
- [ ] Does the test environment use masked data only, or could real data have touched it?
- [ ] Do live parameter changes require two-person approval, and is that enforced in code
      (R4 "High" tier), not just policy?
- [ ] Is there an audit log for every change to money/permissions/audit-tier data?
- [ ] What's the rollback plan if this needs to come back out of PROD?
- [ ] Who is the named support owner once this is live?
```

Human-facing guideline: "This checklist won't get filled in by Claude — it's the list central IT will ask you about anyway. Answering it yourself before you ask for promotion is faster than finding out mid-review that you can't answer it."

R9 — Rename SE_Discipline.md 's pipeline to match ZZ's six named stages

Why: the underlying tools don't need to change, but right now Test_Bed's pipeline (Spec → Plan → Build → Review → Compound → Archive) and ZZ's (Process breakdown → Executable spec → AI build → Business trial → Go-live → Reuse) describe *almost the same six things* under different names, which makes them hard to compare and easy to under-scope. Once R2, R3, R6, R7 exist, the honest renaming is:

#	ZZ's name	Test_Bed step (after R1–R8)	Tool(s)
1	Process breakdown & scope lock	Step 1 — Spec (now requires R2's ticket + named owner)	/openspec-propose
2	Executable spec + TDD	Step 2 — Plan + golden cases (R3) before build	/ce-plan + golden-test authoring
3	AI build + automated tests	Step 3 — Build test-first	/ce-work , Superpowers, karpathy
4	Business trial + fast iteration	Step 4.5 — Business trial (R6, new)	Decision Log (new)
5	Go-live	Step 4 — Review + merge (R4/R5 gates now enforce this)	/ce-code-review , CI gate
6	Reuse	Step 5/6 — Compound + Archive (R7 registry now required)	/ce-compound , /openspec-archive-change

Concrete change: update the stage headers in `SE_Discipline.md` to include the ZZ-style stage name in parentheses, so a reader coming from the ZZ proposal recognizes the mapping immediately, and so gaps are visible by construction (a stage with nothing in its “leaves behind” column is a stage nobody built yet).

R10 — A human-facing “who does what” section (effort + role clarity)

ZZ mechanism: explicit named roles (Finance contacts, Requirement owner, IT contact) each with a stated weekly-hour commitment by project phase (Figure 1) — the human’s job is never ambiguous.

Gap today: Test_Bed’s README says “your only job is to state the goal and answer approve prompts” — true, but it collapses every human role (business owner, domain expert, deployment approver) into one undifferentiated “human,” which works for a solo hobby project and breaks down the moment a Fellow is representing a team.

Concrete change: add a short “Who does what” section to `README.md` Part C:

```
## Who does what (even on a one-person project, wear these hats on purpose)

| Role | When they're needed | What they do |
|---|---|---|
| Requirement/Business owner | Step 1 (R2), Step 4.5 (R6) | Names the rules, approves the frozen spec, judges 1
| Domain/data contact | Step 1 (R2 acceptance criteria), R3 golden cases | Confirms golden-sample answers are 1
| Deployment approver | R8, before PROD promotion | Answers the deployment checklist, owns IT handoff |

On a solo project you play all three – but switch hats on purpose. If you're filling in golden
cases (domain-contact hat) and specifying acceptance criteria (business-owner hat) in the same
five minutes, you may be rubber-stamping your own guess instead of checking it.
```

Human-facing guideline: this section is the guideline — it exists to stop a well-intentioned solo user from silently collapsing “I decided the rule” and “I confirmed the rule is right” into the same mental step.

2. Proposed repo layout after these changes

```
Test_Bed/
├── .claude/
│   ├── skills/openspec-*/          (existing)
│   ├── commands/opsx/*            (existing)
│   └── settings.json               (R4 – risk-tiered permissions + hooks)
├── .github/
│   ├── PULL_REQUEST_TEMPLATE.md    (R5)
│   └── workflows/
│       ├── golden-gate.yml          (R3 – merge-blocking CI)
│       └── golden-drift-check.yml    (R3 – weekly prod-sample re-run)
├── openspec/
│   ├── config.yaml                 (extended – R2 proposal rules)
│   ├── templates/
│   │   └── requirement-ticket.md     (R2)
│   └── changes/...                  (existing)
├── tests/golden/
│   ├── <capability>/...             (R3 – 5-10 cases + exceptions per capability)
│   └── EXCEPTIONS.md
├── scripts/
│   ├── check-golden-coverage.js     (R3)
│   └── warn-if-audit-path.sh        (R4)
├── PROJECT_CONTEXT.md.template      (R1)
├── RISK_TIERS.md                    (R4)
├── DECISION_LOG.md                  (R6, created per active change)
├── CAPABILITY_REGISTRY.md           (R7)
├── DEPLOYMENT_CHECKLIST.md          (R8)
├── CLAUDE.md                        (existing, +1 line: @PROJECT_CONTEXT.md)
├── README.md                       (existing, + "Who does what" – R10)
└── SE_Discipline.md                 (existing, restructured – R9, +Step 4.5 – R6)
```

Everything above is either a **markdown template** (R1, R2, R6, R7, R8, R10 — no runtime dependency, works even before a project has any code) or a **config/CI file** (R4, R5, R3’s CI half) that only activates once a project has actual source and a CI runner. That split matters: Test_Bed today has “no application code yet,” so the templates can be added and validated immediately, while the CI/hook pieces should be added to the **per-project copy** a Fellow makes (per `README.md` Part B), not to Test_Bed’s own root, until Test_Bed itself hosts a reference app to prove them against.

3. Suggested phasing (small-scale echo of ZZ’s five-phase rollout)

ZZ’s proposal phases a harness blueprint → a low-risk pilot → hardening → a second, harder case → a platform. Test_Bed can’t run a business pilot, but it can phase *validating these recommendations* the

same way, so the templates aren't just written once and left unverified:

1. **Phase 1 — Templates only (R1, R2, R6, R7, R8, R10).** Pure markdown, no tooling risk. Add them, update `README.md` / `SE_Discipline.md` to reference them, done in an afternoon.
2. **Phase 2 — Prove the pipeline gets a business-owner gate.** Run one real `/openspec-propose` through the new `requirement-ticket.md` template end-to-end and confirm the "Freeze" concept actually stops `/ce-plan` from starting while a "Blocks build" item is open — this is a discipline check, not a code check. (`/ce-plan` is installed per §0; confirm it autocompletes in a fresh session before relying on it here.)
3. **Phase 3 — Add a first reference app** (even a toy one) so R3's golden-test CI gate and R4's `settings.json` /hooks have something real to enforce against, rather than living as untested templates.
4. **Phase 4 — Verify the CI gate actually blocks a bad merge** (a deliberately broken PR should fail `golden-gate.yml`) and that a hook actually stops an edit to a `deny` -listed path.
5. **Phase 5 — Update `Compare_to_ZZ.md` 's verdict table** once all five rows move from "Missing" to "Present" — that comparison document is the natural place to track whether these recommendations actually closed the gap, and should be re-run rather than left as a one-time snapshot.

4. Deliberately not imported from ZZ's proposal

These are real and valuable in ZZ's proposal, but they are staffing/organizational choices for a paid delivery engagement, not something a self-serve tooling repo should prescribe:

- The 3-person delivery-unit model and 10-person org structure.
- Named accountable leads per throughline with explicit external hour/week asks.
- A five-phase *client* rollout tied to two specific business cases (PM comps → Trading).
- Phased capability transfer (Shadow → Guided → Solo) with a coaching role.

What a Fellow should do instead: R10's "who does what" table is the intentionally lightweight substitute — it keeps the *role separation* ZZ's staffing model protects (business owner ≠ domain confirmer ≠ deployment approver) without prescribing headcount. If a Fellow is actually leading a multi-person effort, ZZ's staffing structure is worth reading directly rather than approximating through `Test_Bed`.

5. Next three actions

1. **Verify the install fires in a live session (§0's remaining caveat)** — open a fresh Claude Code session in this repo and confirm `/plugins` lists all three, `/ce-code-review` / `/ce-plan` / `/ce-work` / `/ce-compound` autocomplete, and a Superpowers TDD skill plus karpathy-guidelines appear in that session's skill listing. Five minutes, and it's the last unverified link between "installed" and "the recommendations below actually work as written."
2. Add the six pure-markdown templates from Phase 1 (§3) — R1, R2, R6, R7, R8, R10 — and wire the two one-line references into `CLAUDE.md` and `README.md` .

3. Pick or scaffold a small reference app inside Test_Bed so R3 (golden-test CI gate) and R4 (settings.json + hooks) can be built and *proven to actually block a bad merge*, rather than shipped as templates nobody has exercised.